Difference between Git and GitHub?

Answer :

- Git → Version control tool (local system)
- GitHub → Cloud-based platform for hosting Git repositories
- Git works offline; GitHub requires internet
- Git handles versioning; GitHub adds collaboration features (PR, issues)

One-liner:
Git is a tool, while GitHub is a platform to host and collaborate on Git repositories.

What is staging area?

Answer :

- Intermediate area before commit
- Allows selective commit
- Managed using `git add`
- Helps organize changes

One-liner:
The staging area is where changes are prepared before committing to the repository.

What is commit?

Answer :

- Snapshot of changes
- Saved in Git history
- Requires message (`git commit -m`)
- Each commit has unique ID (SHA)

One-liner:
A commit is a snapshot of your staged changes saved in the Git history.

What is merge?

Answer :

- Combines changes from different branches
- Performed using `git merge`
- Can create merge conflicts
- Preserves history

One-liner:
Merge combines changes from one branch into another.

What is rebase?

Answer :

- Re-applies commits on top of another branch
- Creates linear history
- Cleaner than merge
- Can rewrite history

One-liner:
Rebase moves your branch on top of another branch to create a cleaner history.

Difference between merge and rebase?

Answer :

- Merge → Keeps history, creates merge commit
- Rebase → Rewrites history, no extra commit
- Merge safer for shared branches
- Rebase cleaner for local work

One-liner:
Merge preserves history, while rebase rewrites it for a cleaner commit flow.

What is Git stash?

Answer :

- Temporarily saves uncommitted changes
- Keeps working directory clean
- Use `git stash` and `git stash pop`
- Useful when switching branches

One-liner:
Git stash temporarily saves changes without committing them.

What is Git revert?

Answer :

- Creates a new commit to undo changes
- Safe for shared branches
- Does not delete history

One-liner:
Git revert safely undoes changes by creating a new commit.

What is merge conflict?

Answer :

- Happens when same file is modified in multiple branches
- Git cannot auto-merge
- Requires manual resolution
- Marked in file

One-liner:
A merge conflict occurs when Git cannot automatically combine changes.

What is Git workflow?

Answer :

- Process of using Git effectively
- Common workflows:
    - Feature branching
    - Gitflow
- Improves collaboration

One-liner:
Git workflow defines how teams collaborate using Git.

How do you resolve merge conflicts?

Answer :

- Identify conflict files
- Edit manually
- Remove conflict markers

- Add and commit changes

One-liner:
Merge conflicts are resolved by manually editing and committing the corrected code.

What is Git fetch?

Answer :

- Downloads changes from remote repository
- Does NOT merge automatically
- Updates remote tracking branches
- Safer than pull for review

One-liner:
Git fetch retrieves changes from remote without merging them.

Difference between fetch and pull?

Answer :

- Fetch → Only downloads changes
- Pull → Fetch + Merge
- Fetch gives control before merging
- Pull is quicker but less safe

One-liner:
Fetch only downloads changes, while pull downloads and merges them.

What is Git cherry-pick?

Answer :

- Picks specific commit from another branch
- Applies it to current branch
- Useful for hotfixes
- Command: `git cherry-pick <commit-id>`

One-liner:
Cherry-pick applies a specific commit from one branch to another.

What is Git diff?

Answer :

- Shows differences between files/commits
- Helps track changes
- Can compare branches
- Used before commit

One-liner:
Git diff shows changes between commits, branches, or files.

What is Git squash?

Answer :

- Combines multiple commits into one
- Cleans commit history
- Done using rebase
- Useful before merge

One-liner:
Squash combines multiple commits into a single commit.

What is upstream branch?

Answer :

- Tracks remote branch
- Helps with push/pull
- Set using `-u` flag

One-liner:
An upstream branch links your local branch to a remote branch.

How do you handle a situation where you pushed wrong code to main branch?

Answer :

- Identify commit using `git log`
- Use `git revert` (safe way)
- Avoid force push in shared repo
- Communicate with team

One-liner:
I use git revert to safely undo changes without affecting history.

What is Git workflow you follow?

Answer :

- Feature branch workflow
- Pull latest code
- Create branch
- Commit changes
- Push and create PR
- Code review + merge

One-liner:
I follow feature branching with PR-based reviews for safe and collaborative development.

What is pull request (PR)?

Answer :

- Request to merge code
- Used for code review
- Ensures quality
- Common in teams

One-liner:
A pull request is a proposal to merge changes with code review.

How do you undo last commit?

Answer :

- `git reset --soft HEAD~1` → keep changes
- `git reset --hard HEAD~1` → delete changes
- `git revert` → safe undo

One-liner:
I use reset or revert depending on whether I want to keep changes or not.

What is Git branching strategy?

Answer :

- Gitflow
- Feature branching
- Trunk-based development
- Depends on team size

One-liner:

I choose branching strategy based on team size and release frequency.

Your branch is behind main, what will you do?

Answer :

- Fetch latest code
- Rebase or merge main
- Resolve conflicts
- Push updated branch

One-liner:

I update my branch using rebase or merge to sync with main.

Production bug fix urgently?

Answer :

- Create hotfix branch
- Fix issue
- Test quickly
- Merge to main
- Deploy

One-liner:

I create a hotfix branch, fix the issue, and deploy quickly.

How do you ensure code quality?

Answer :

- Pull requests
- Code review
- CI pipelines
- Automated testing

One-liner:

I ensure quality through PR reviews and automated CI testing.

When interviewer asks anything tricky, say:

“I would first analyze the situation using Git logs and history, then choose the safest approach like revert or rebase depending on whether the branch is shared or not